MongoDB Assignment

0. Structure

The structure of the project is the following:

- **shopnet.js**: The main execution script that initializes the database, creates indexes, runs queries, and executes aggregation pipelines.
- **seedData.js**: Contains all seed data and inserts documents into collections in the correct order (vendors → products → customers → orders). It is called in **shopnet.js**.
- **README.md / report.pdf**: The report describing schema design, queries, and results.
- **monthly_sales_trend.png**: Visual representation of the monthly sales aggregation.
- **plot.py**: Python script used to generate charts for the report.

1. Introduction

In this assignment I design, populate, and query a MongoDB database that models a simplified e-commerce platform. I practice the full lifecycle of a MongoDB application: schema design for a document-oriented store, bulk and conditional data ingestion, expressive query construction, and multi-stage aggregation pipelines.

2. Schema Design

The database for ShopNest consists of four main collections: `products`, `customers`, `orders`, and `vendors`. Each collection is designed following MongoDB's document-oriented model, with a focus on performance, readability, and real-world e-commerce requirements.

- vendors: stores marketplace vendors.
- products: stores product listings and references vendors through vendor_id.
- customers: stores customer profile and address information.
- orders: stores customer purchases, embedding order line items inside each order document.

Products Collection

```
{
  _id: ObjectId,            // auto-generated
  sku: String,              // unique, e.g. "SNT-1001"
  name: String,
  category: String,         // "Electronics" | "Books" | "Clothing" | ...
  vendor_id: ObjectId,      // references vendors._id
  price: NumberDecimal,
  stock: NumberInt,
  ratings: {
    average: Number,        // 1.0 – 5.0
    count: NumberInt
  },
  tags: [String],
  created_at: Date
}
```

Customers Collection

```
{
  _id: ObjectId,
  email: String,            // unique
  name: String,
  address: {
    city: String,
    country: String
  },
  tier: String,             // "bronze" | "silver" | "gold"
  joined_at: Date
}
```

Orders Collection

```
{
  _id: ObjectId,
  customer_id: ObjectId,    // references customers._id
  status: String,           // "pending" | "shipped" | "delivered" | "cancelled"
  items: [                  // embedded array — no join needed for totals
    {
      product_id: ObjectId,
      sku: String,
      name: String,         // repeated for read performance
      qty: NumberInt,
      unit_price: NumberDecimal
    }
  ],
  total: NumberDecimal,     // pre-computed sum of items
  created_at: Date
}
```

Vendors Collection

```
{
  _id: ObjectId,
  name: String,
  email: String,
  country: String,
  created_at: Date
}
```

3. Embedding Decision Justification

In this design, order line items are embedded directly inside the `orders` collection rather than stored in a separate `order_items` collection. This decision is primarily driven by the expected read-heavy access pattern of an e-commerce system. In most cases, when an order is retrieved, all of its associated items are needed together (e.g., for order history or invoice display). Embedding eliminates the need for joins, improving read performance and simplifying queries such as total calculation.

Regarding document size, MongoDB enforces a 16MB BSON document limit. In this application, each order is expected to contain a reasonable number of items, so this limit is unlikely to be exceeded.

However, if order items were frequently updated independently (e.g., changing quantities, prices, or availability after the order is created) or if each order could contain a very large number of items, embedding would become inefficient. In such cases, storing order items in a separate `order_items` collection would be preferable, as it would reduce document size, avoid frequent rewriting of large documents, and provide better flexibility and scalability for write-heavy operations.

4. Indexes

Indexes were created to improve query performance and enforce data integrity where necessary.

- Unique Index on Product SKU

```
db.products.createIndex({ sku: 1 }, { unique: true });
```

This index ensures that each product has a unique SKU, preventing duplicate product entries. Additionally, it enables fast lookups when searching for products by SKU, which is a common operation in e-commerce systems.

- Compound Index on Orders (Customer + Date)

```
db.orders.createIndex({ customer_id: 1, created_at: -1 });
```

This compound index supports efficient retrieval of a customer's order history. By indexing customer_id and sorting by created_at in descending order, it allows queries to quickly return the most recent orders for a specific customer without requiring additional sorting in memory.

- Index on Order Status

```
db.orders.createIndex({ status: 1 });
```

This index optimizes queries that filter orders based on their status (e.g., "pending", "shipped", "delivered", "cancelled"). It is particularly useful for dashboard views and administrative tools that frequently group or filter orders by status.

- Full-Text Index on Products

```
db.products.createIndex({ name: "text", tags: "text", category: "text" });
```

This text index enables full-text search functionality across multiple fields. It allows users to search for products using keywords (e.g., "wireless headphones") and returns results ranked by relevance using MongoDB's text scoring mechanism.

5. Seed Data

The data includes:

- **Vendors**: The dataset includes 3 vendors from different countries
- **Products**: The dataset includes 20 products across 5 categories:
    - Electronics

    - Books
    - Clothing
    - Home
    - Wellness

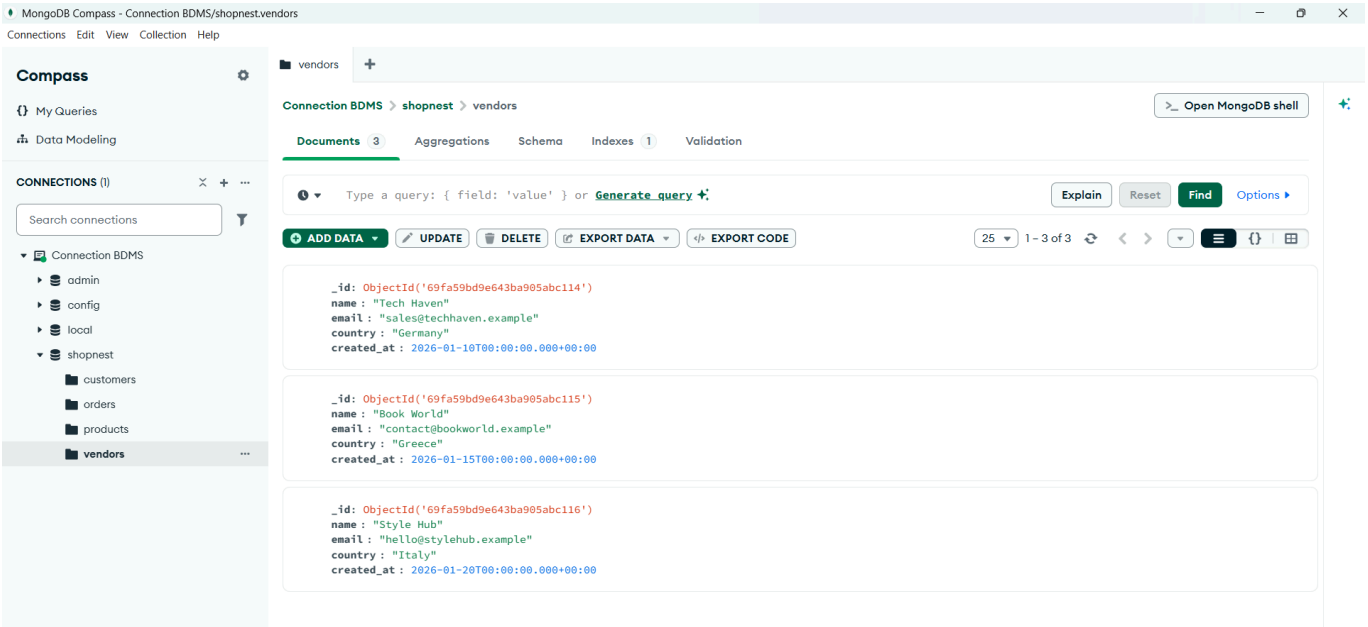
- **Customers**: The dataset includes 10 customers from multiple countries, with a mix of bronze, silver, and gold membership tiers.
- **Orders**: The dataset includes 30 orders with different statuses: pending, shipped, delivered, and cancelled.
    - The orders reference real customer IDs and contain embedded line items that reference real products.
    - One customer has more than 5 orders, and at least 5 orders are marked as cancelled.

Insert

The order of inserts matters: Vendors → Products → Customers → Orders

Because:

- Products reference real Vendor ids.
- Orders will reference real Customer and Product ids.



Conditional insert with duplicate detection

To prevent duplicate products based on the `sku` field, a custom helper function `insertProductSafe` is implemented.
Instead of directly inserting documents, it uses an **upsert operation** to insert a product only if it does not already exist.

Afterwards, the function is tested by attempting to insert two products with the same SKU.

Bulk price update

A bulk update operation is used to apply a **10% discount** to all products in the *Electronics* category that have stock greater than 50.

This operation uses the $mul update operator to efficiently multiply the price field by 0.90, reducing it by 10%.

The result object provides useful feedback:

- matchedCount: number of documents that met the criteria
- modifiedCount: number of documents that were actually updated

Using updateMany allows all matching documents to be updated in a single operation, improving performance compared to updating documents individually.

6. Queries

```
======================================================
Query 1: Books with good ratings and in stock, sorted by price
======================================================
Description:
This query returns available Books products with an average rating of at least 4.0
and stock greater than 0, sorted by price in ascending order.

First 3 result documents:
[
 {
   name: 'Mystery Novel',
   price: Decimal128('12.99'),
   ratings: {
     average: 4.3
   },
   tags: [
     'fiction',
     'new-arrival'
   ]
 },
 {
   name: 'JavaScript Basics',
   price: Decimal128('19.99'),
   ratings: {
     average: 4
   },
   tags: [
     'programming',
     'sale'
   ]
 },
 {
   name: 'MongoDB Practical Guide',
   price: Decimal128('25.99'),
   ratings: {
     average: 4.5
```

```
    },
    tags: [
      'database',
      'featured',
      'new-arrival'
    ]
  }
]
```

```
=====================================================
Query 2: Products tagged with both new-arrival and featured
=====================================================
Description:
This query returns products that contain both the new-arrival and featured tags.

First 3 result documents:
[
 {
   sku: 'SNT-1001',
   name: 'Wireless Headphones',
   category: 'Electronics',
   price: Decimal128('80.9910'),
   tags: [
     'wireless',
     'headphones',
     'featured',
     'new-arrival'
   ]
 },
 {
   sku: 'SNT-2001',
   name: 'MongoDB Practical Guide',
   category: 'Books',
   price: Decimal128('25.99'),
   tags: [
     'database',
     'featured',
     'new-arrival'
   ]
 },
 {
   sku: 'SNT-3003',
   name: 'Running Sneakers',
   category: 'Clothing',
   price: Decimal128('74.99'),
   tags: [
     'shoes',
     'new-arrival',
     'featured'
   ]
```

```
 }
]
```

```
======================================================
Query 2B: Products tagged with either clearance or sale
======================================================
Description:
This query returns products that contain either the clearance tag or the sale tag.

First 3 result documents:
[
 {
   sku: 'SNT-1002',
   name: 'Bluetooth Speaker',
   category: 'Electronics',
   price: Decimal128('53.9910'),
   tags: [
     'wireless',
     'audio',
     'sale'
   ]
 },
 {
   sku: 'SNT-1004',
   name: 'Mechanical Keyboard',
   category: 'Electronics',
   price: Decimal128('98.9910'),
   tags: [
     'keyboard',
     'gaming',
     'featured',
     'sale'
   ]
 },
 {
   sku: 'SNT-2002',
   name: 'JavaScript Basics',
   category: 'Books',
   price: Decimal128('19.99'),
   tags: [
     'programming',
     'sale'
   ]
 }
]
```

```
======================================================
Query 3: Full-text search for wireless headphones
======================================================
```

```
Description:
This query performs a full-text search for products matching wireless headphones
and sorts the results by text relevance score.

First 3 result documents:
[
 {
   name: 'Wireless Headphones',
   price: Decimal128('80.9910'),
   score: 3.6
 },
 {
   name: 'Bluetooth Speaker',
   price: Decimal128('53.9910'),
   score: 1.1
 }
]
```

```
======================================================
Query 4: Gold customers from Greece or Germany
======================================================
Description:
This query returns gold-tier customers whose country is either Greece or Germany.

First 3 result documents:
[
 {
   email: 'eleni.papadopoulou@example.com',
   name: 'Eleni Papadopoulou',
   address: {
     city: 'Athens',
     country: 'Greece'
   }
 },
 {
   email: 'anna.mueller@example.com',
   name: 'Anna Mueller',
   address: {
     city: 'Berlin',
     country: 'Germany'
   }
 }
]
```

```
======================================================
Query 5: Orders containing at least one line item with qty > 3
======================================================
Description:
This query returns orders that contain at least one embedded line item with a
```

```
quantity greater than 3.

First 3 result documents:
[
 {
   _id: ObjectId('69fa59bd9e643ba905abc13f'),
   customer_id: ObjectId('69fa59bd9e643ba905abc119'),
   status: 'delivered',
   items: [
     {
       product_id: ObjectId('69fa59bd9e643ba905abc132'),
       sku: 'SNT-4003',
       name: 'Reusable Water Bottle',
       qty: 4,
       unit_price: Decimal128('11.99')
     }
   ],
   total: Decimal128('47.96'),
   created_at: ISODate('2026-04-12T08:45:00.000Z')
 },
 {
   _id: ObjectId('69fa59bd9e643ba905abc150'),
   customer_id: ObjectId('69fa59bd9e643ba905abc11e'),
   status: 'delivered',
   items: [
     {
       product_id: ObjectId('69fa59bd9e643ba905abc132'),
       sku: 'SNT-4003',
       name: 'Reusable Water Bottle',
       qty: 5,
       unit_price: Decimal128('11.99')
     }
   ],
   total: Decimal128('59.95'),
   created_at: ISODate('2026-03-22T09:30:00.000Z')
 }
]
```

```
======================================================
Query 6: Delivered orders in the last 30 days
======================================================
Description:
This query returns delivered orders placed within the last 30 days of the seed
dataset, sorted from newest to oldest.

First 3 result documents:
[
 {
   _id: ObjectId('69fa59bd9e643ba905abc135'),
   customer_id: ObjectId('69fa59bd9e643ba905abc117'),
   status: 'delivered',
```

```
    items: [
      {
        product_id: ObjectId('69fa59bd9e643ba905abc121'),
        sku: 'SNT-1001',
        name: 'Wireless Headphones',
        qty: 1,
        unit_price: Decimal128('89.99')
      },
      {
        product_id: ObjectId('69fa59bd9e643ba905abc125'),
        sku: 'SNT-1005',
        name: 'USB-C Hub',
        qty: 2,
        unit_price: Decimal128('24.99')
      }
    ],
    total: Decimal128('139.97'),
    created_at: ISODate('2026-04-30T10:00:00.000Z')
  },
  {
    _id: ObjectId('69fa59bd9e643ba905abc146'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11b'),
    status: 'delivered',
    items: [
      {
        product_id: ObjectId('69fa59bd9e643ba905abc123'),
        sku: 'SNT-1003',
        name: 'Smartwatch',
        qty: 1,
        unit_price: Decimal128('149.99')
      }
    ],
    total: Decimal128('149.99'),
    created_at: ISODate('2026-04-29T14:30:00.000Z')
  },
  {
    _id: ObjectId('69fa59bd9e643ba905abc13b'),
    customer_id: ObjectId('69fa59bd9e643ba905abc118'),
    status: 'delivered',
    items: [
      {
        product_id: ObjectId('69fa59bd9e643ba905abc121'),
        sku: 'SNT-1001',
        name: 'Wireless Headphones',
        qty: 2,
        unit_price: Decimal128('89.99')
      }
    ],
    total: Decimal128('179.98'),
    created_at: ISODate('2026-04-28T10:10:00.000Z')
  }
]
```

6. Aggregation Pipelines

```
======================================================
Aggregation 1: Revenue by category
======================================================
Description:
This aggregation computes total delivered-order revenue and the number of distinct
products sold for each product category.

Aggregation results:
[
  {
    total_revenue: Decimal128('639.92'),
    category: 'Electronics',
    products_sold: 5
  },
  {
    total_revenue: Decimal128('494.92'),
    category: 'Clothing',
    products_sold: 4
  },
  {
    total_revenue: Decimal128('292.87'),
    category: 'Home',
    products_sold: 3
  },
  {
    total_revenue: Decimal128('205.91'),
    category: 'Books',
    products_sold: 5
  },
  {
    total_revenue: Decimal128('52.98'),
    category: 'Wellness',
    products_sold: 2
  }
]
```

```
======================================================
Aggregation 2: Customer lifetime value
======================================================
Description:
This aggregation calculates each customer's order count, delivered-order spending,
average order value, and favourite delivered-order category, then returns the top
10 customers by spending.
```

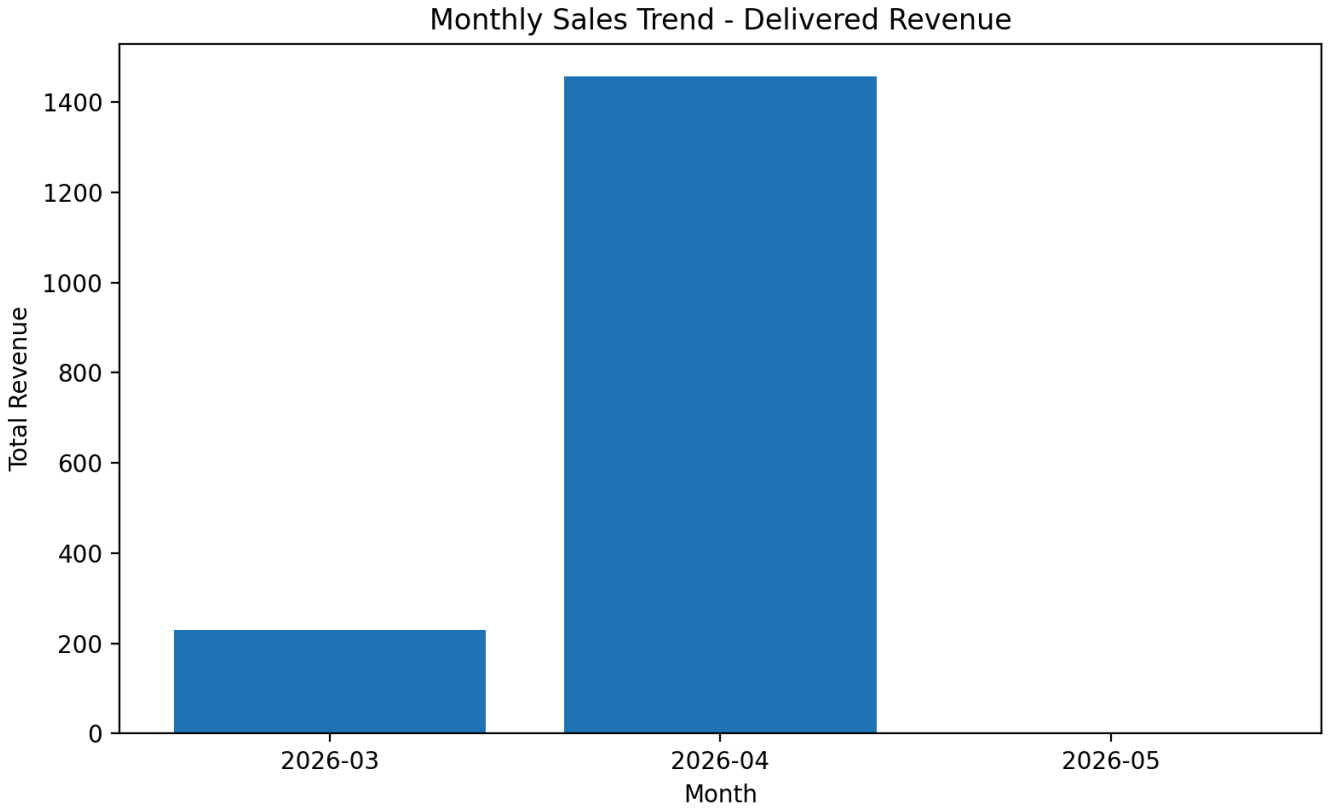
```
Aggregation results:
[
  {
    total_orders: 4,
    total_spent: Decimal128('294.96'),
    customer_id: ObjectId('69fa59bd9e643ba905abc118'),
    customer_name: 'Nikos Georgiou',
    avg_order_value: Decimal128('106.86'),
    favourite_category: 'Clothing'
  },
  {
    total_orders: 3,
    total_spent: Decimal128('289.96'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11d'),
    customer_name: 'Luca Rossi',
    avg_order_value: Decimal128('105.32'),
    favourite_category: 'Clothing'
  },
  {
    total_orders: 6,
    total_spent: Decimal128('265.94'),
    customer_id: ObjectId('69fa59bd9e643ba905abc117'),
    customer_name: 'Eleni Papadopoulou',
    avg_order_value: Decimal128('89.81'),
    favourite_category: 'Electronics'
  },
  {
    total_orders: 3,
    total_spent: Decimal128('229.97'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11b'),
    customer_name: 'Peter Schmidt',
    avg_order_value: Decimal128('103.32'),
    favourite_category: 'Home'
  },
  {
    total_orders: 4,
    total_spent: Decimal128('208.95'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11a'),
    customer_name: 'Anna Mueller',
    avg_order_value: Decimal128('78.48'),
    favourite_category: 'Books'
  },
  {
    total_orders: 2,
    total_spent: Decimal128('129.93'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11e'),
    customer_name: 'Isabel Garcia',
    avg_order_value: Decimal128('64.96'),
    favourite_category: 'Home'
  },
  {
    total_orders: 3,
    total_spent: Decimal128('77.94'),
```

```
    customer_id: ObjectId('69fa59bd9e643ba905abc119'),
    customer_name: 'Maria Christou',
    avg_order_value: Decimal128('37.64'),
    favourite_category: 'Home'
  },
  {
    total_orders: 1,
    total_spent: Decimal128('70.97'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11f'),
    customer_name: 'Kostas Dimitriou',
    avg_order_value: Decimal128('70.97'),
    favourite_category: 'Books'
  },
  {
    total_orders: 3,
    total_spent: Decimal128('65.00'),
    customer_id: ObjectId('69fa59bd9e643ba905abc11c'),
    customer_name: 'Sofia Martin',
    avg_order_value: Decimal128('65.32'),
    favourite_category: 'Books'
  },
  {
    total_orders: 1,
    total_spent: Decimal128('52.98'),
    customer_id: ObjectId('69fa59bd9e643ba905abc120'),
    customer_name: 'Emma Jansen',
    avg_order_value: Decimal128('52.98'),
    favourite_category: 'Wellness'
  }
]
```

```
======================================================
Aggregation 3: Monthly sales trend
======================================================
Description:
This aggregation groups orders by month and computes delivered-order revenue,
order count, and average order value for each month in the dataset.

Aggregation results:
[
  {
    total_revenue: Decimal128('229.91'),
    number_of_orders: 5,
    month: '2026-03',
    avg_order_value: Decimal128('81.98')
  },
  {
    total_revenue: Decimal128('1456.69'),
    number_of_orders: 22,
    month: '2026-04',
    avg_order_value: Decimal128('88.66')
```

```
  },
  {
    total_revenue: Decimal128('0'),
    number_of_orders: 3,
    month: '2026-05',
    avg_order_value: Decimal128('36.16')
  }
]
```




```
============================================================
Aggregation 4: Vendor performance report
============================================================
Description:
This aggregation joins delivered orders with products and vendors to calculate
product listing counts, units sold, gross revenue, and the best-selling product
for each vendor.

Aggregation results:
[
  {
    vendor_name: 'Style Hub',
    vendor_email: 'hello@stylehub.example',
    total_units_sold: 23,
    gross_revenue: Decimal128('840.77'),
    top_product: 'Reusable Water Bottle',
    total_products_listed: 10
  },
  {
```

```
    vendor_name: 'Tech Haven',
    vendor_email: 'sales@techhaven.example',
    total_units_sold: 8,
    gross_revenue: Decimal128('639.92'),
    top_product: 'Wireless Headphones',
    total_products_listed: 6
  },
  {
    vendor_name: 'Book World',
    vendor_email: 'contact@bookworld.example',
    total_units_sold: 11,
    gross_revenue: Decimal128('205.91'),
    top_product: 'Mystery Novel',
    total_products_listed: 5
  }
]
```

8. Prerequisities

Installation

To run this project, the following tools must be installed:

- **MongoDB Server** (running locally)
- **mongosh (MongoDB Shell)**
- (Optional) **MongoDB Compass** for visual inspection of the database
- (Optional) **Visual Studio Code** for editing and running scripts

Ensure that MongoDB is running and that `mongosh` is available in your system PATH.

Project Setup

Ensure that all project files are placed in the same directory "MongoDB"

Execution

To initialize the database, insert data, and run all queries and aggregations, execute the following command:

```
mongosh shopnet.js
```